

Model selection and evaluation

Quick recall:

Overfitting: when a model memorizes training data but fails on new data

- Small k in k -NN = **overfitting** (too sensitive to noise)
- Large k = **underfitting** (too smooth, misses patterns)

Challenge:

How do we systematically find the right k ?

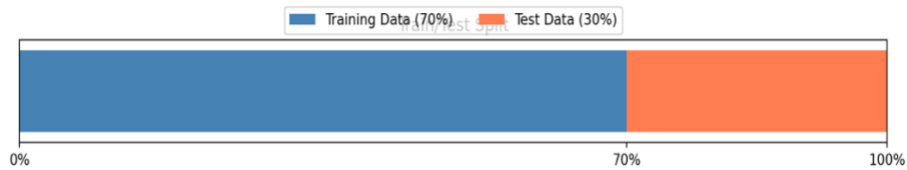
Key Question:

How do we actually measure whether our model generalizes?

Train/Test Split

We can't evaluate on data we trained on — of course it does well on that!

Solution: Hold out some data for testing.

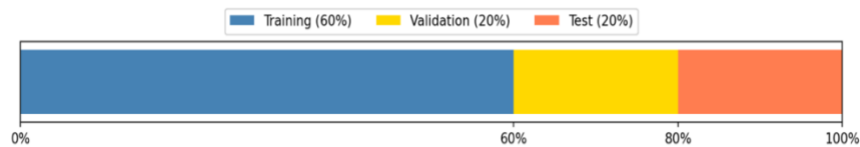


- *Train* the model on the ***training data*** only
- *Evaluate* on the held-out ***test data***
- ***Test accuracy*** tells us how well the model generalizes

Train/Validation/Test Split

What if we tune k using the test set? We might overfit to the test set!

Solution: Three-way split



- **Train:** fit the model
- **Validation:** tune hyperparameters (like k)
- **Test:** final evaluation (touch only once!)

Why keep test data separate?

If you repeatedly check test accuracy while tuning, you're indirectly fitting to the test set.

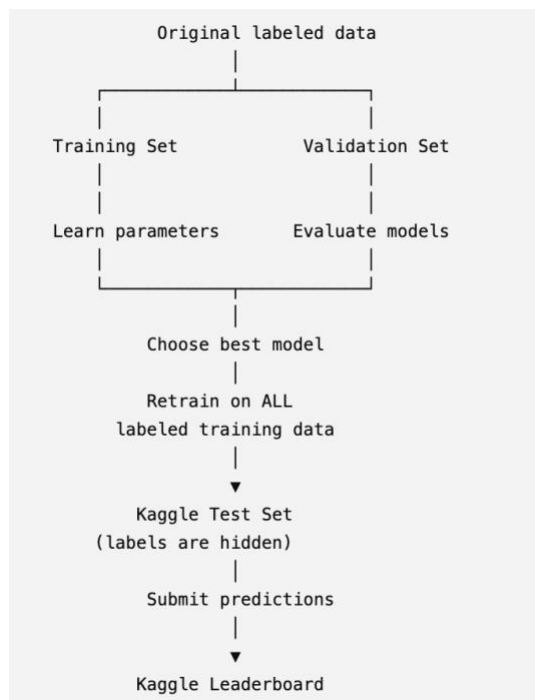
Example: Train/Validate/Test

Kaggle competitions illustrate this perfectly:

- **Public leaderboard:** you can submit and see your score (like validation)
- **Private leaderboard:** revealed only at the end (the true test)

Suppose Kaggle gives you **10,000 labeled houses**. If you train on all 10,000, how do you know whether your model is actually good?

You don't. Instead, you pretend that some of the answers are unknown. (say 8000-2000 split!)



Why retrain on all the labeled data?

After you've used the validation set to choose the best model, you no longer need to keep those examples separate. Since their labels are known, you can retrain the chosen model on **all 10,000 labeled houses** before making predictions on Kaggle's test set. This gives the model more data to learn from and often improves performance.

Example: Train/Validate/Test

Think of a Kaggle competition like preparing for a final exam:

- **Training set:** Homework and practice problems. You learn from these because the answers are provided.
- **Validation set:** Practice exam. You know the answers afterward, so you can identify mistakes and improve your study strategy.
- **Test set (Kaggle):** The actual final exam. You never see the answers—you only receive your score.

Think in terms of:

- **Training** is for **learning**.
- **Validation** is for **choosing and tuning** the model.
- **Test** is for **final evaluation** on completely unseen data.

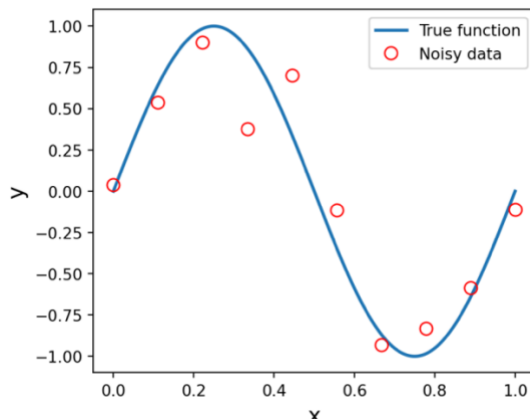
- You have 100 samples. What's wrong with using 95% for training and 5% for testing?
- A colleague says "I tried 20 different values of k and picked the one with the best test accuracy." What's the problem?

Understanding overfitting: Polynomial Regression example

Let's use polynomial regression to build intuition about overfitting.

This example makes overfitting very visible — then we'll apply the same ideas to k-NN.

Consider synthetic data generated from $y = \sin(2\pi x)$ plus noise:



We'll try to fit polynomials of different degrees:

$$y(x, \mathbf{w}) = w_0 + w_1 x^1 + w_2 x^2 + \dots + w_k x^k$$

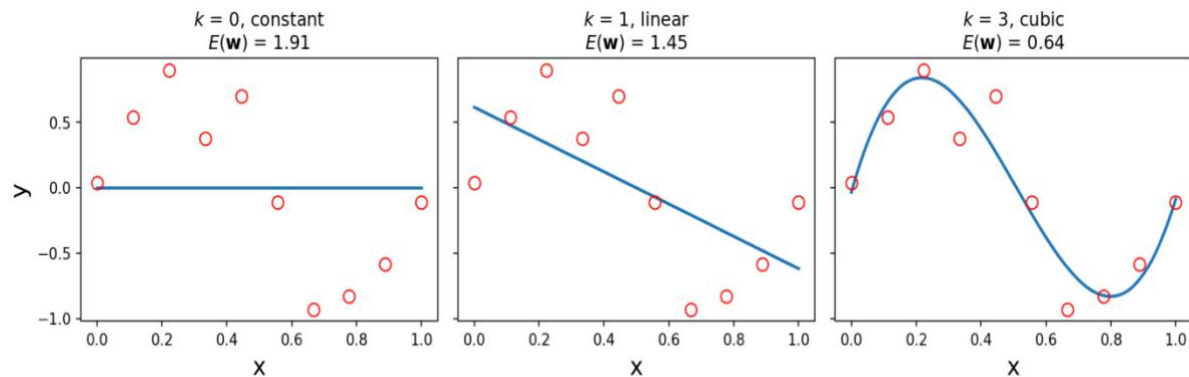
Here:

- k is the hyperparameter (we choose it)
- The w_i values are parameters (learned from the data)

Understanding overfitting: Polynomial Regression example

Model selection: Choosing k

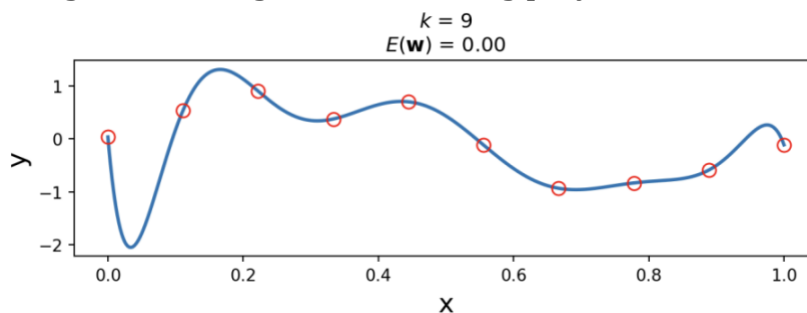
The problem of choosing the hyperparameter k is called model selection. Let's apply regression to build the best possible constant (order 0), linear (order 1), and cubic (order 3) models.



Here $E(\mathbf{w})$ is the **training error**.

A third-order polynomial ($k = 3$) leads to small training error and hence is a good fit. What about higher order polynomials?

By setting $k = 9$, we get the following polynomial to fit the data:



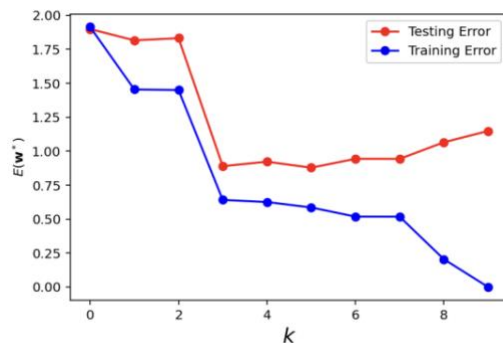
Is the 9th order polynomial a "better" model for this dataset?

Understanding overfitting: Polynomial Regression example

Model selection: Choosing k

We want the model to have a good generalization ability (i.e., do a good job of predicting on future data).

The 9th degree polynomial has poor generalization ability for this dataset.



Test data is data that was not used to train the model but comes from the same source.

As polynomial order increases:

- **Training error** always *decreases* (eventually hits zero)
- **Test error** *decreases*, then *increases* again

The best test error is at $k = 3$ (cubic). Higher-order polynomials **overfit**

This is the core tradeoff: **fitting** training data well vs. **generalizing** to new data.

Two ways to look at it:

- Too many parameters for the amount of data: We can see this in the fact that we have only 10 training points, and the 9th order polynomial has 10 coefficients.
- Model too complex for the underlying pattern — the model is not just fitting the underlying phenomenon, but also the noise in the data.

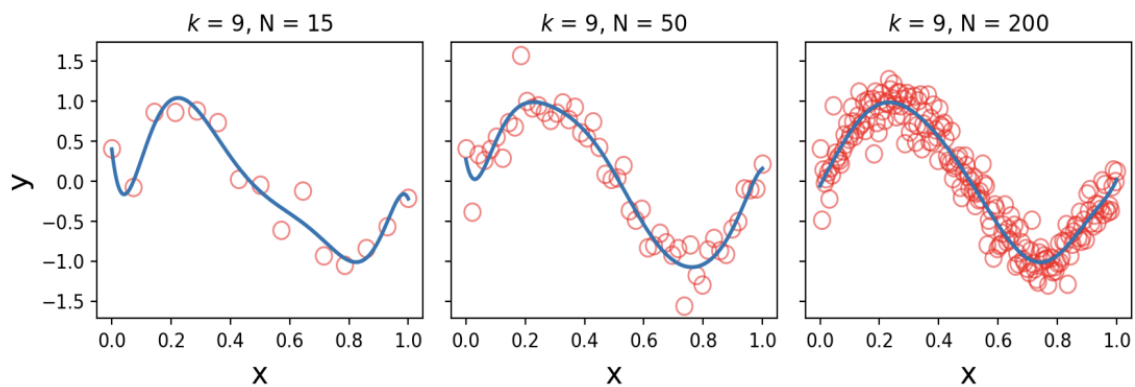
These suggest techniques we may use to avoid overfitting:

- Increase the amount of training data. All else being equal, more training data is always better
- Limit the complexity of the model. Model complexity is often controlled via hyperparameters.

More Training Data

It's not necessarily true that an order-3 polynomial is best for this problem. But the higher the order of polynomial we want to fit, the more data we need to avoid overfitting.

Here we use an order-9 polynomial for increasing amounts of training data ($N = 15, 50, 200$):



Hyperparameters

Getting more data helps but we can't always get more data!

Instead: we *control model complexity*!

A hyperparameter is a setting we choose before training:

- Polynomial degree k
- Number of neighbors k in k -NN
- Learning rate, regularization strength, ...

How do we choose the right value? *Hold out validation data and compare!*

But

A single random split can be lucky or unlucky!

Solution 1: Random Subsampling — do multiple random splits and average the results.

Solution 2: Cross-Validation — systematically use all data for both training and testing.

Random Subsampling

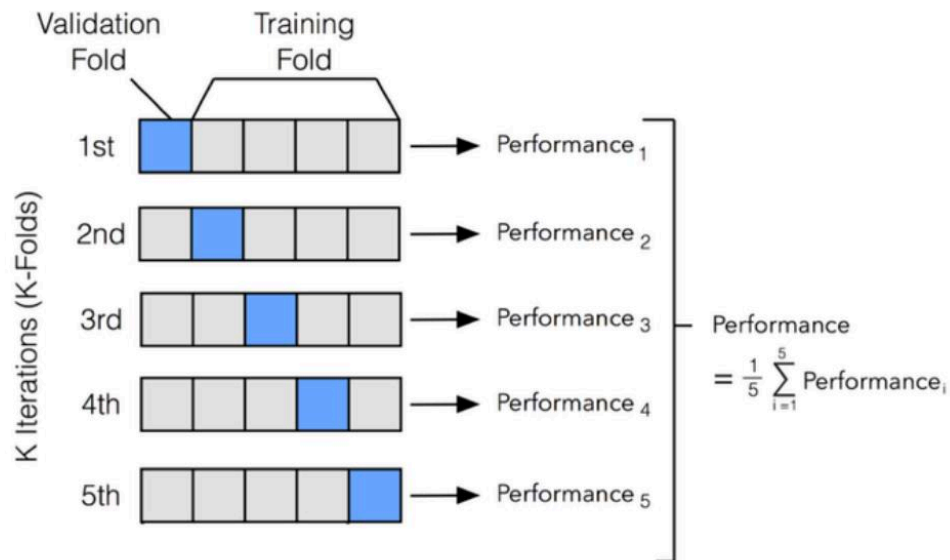
For each hyperparameter value:

- Randomly split data 5+ times
- Train and evaluate on each split
- Average the results

More robust than a single split — but there's an even better approach...

Cross Validation

k-fold cross-validation: split data into parts. Each part takes a turn as a validation set.



The value of k can vary up to the size of the dataset.

The ***larger k*** we use, the more data is used for training, but the more folds must be evaluated, which ***increases the time*** required.

In the extreme case where ***k is equal to the data size***, then each data item is held out by itself; this is called "***leave-one-out***".

Common defaults: 5-fold, 10-fold: good balance!

You're doing 5-fold cross-validation on a dataset of 1000 samples (assume you've already set aside a test set of 200, and you have 1000 left).

1. How many samples are in each fold?
2. How many times do you train the model?
3. How many samples does each model see during training?

Grid search with cross validation

Suppose we use 5-fold cross validation. So, our data is divided into 5 equal pieces.

Eventually every fold will become the validation set once.

Suppose for $k = 3$

Fold	Accuracy %
1	94
2	96
3	95
4	93
5	97

Average accuracy: 95%

Now, suppose for $k = 5$

Fold	Accuracy %
1	96
2	98
3	97
4	95
5	96

Average accuracy: 96.4%

Repeat for every candidate value of k .

Grid: Simply means all combinations of hyper parameters you want to try.

For our case, it may be all our candidate values of k : 1,3,5,7,9

Grid search + cross validation:

For every hyperparameter combination → perform 5-fold cross validation →

Compute average validation accuracy → Keep the best one

Evaluating a binary classifier

Suppose we run the k -nearest neighbors algorithm on some dataset. How do we measure whether it has performed well or poorly?

That is: how do we evaluate a classifier?

In the simple case of a *binary classifier*-- meaning that there are exactly two possible labels -- we can call one class the '*Positive*' class and one the '*Negative*' class.

The most basic measure of success for a classifier is *accuracy*: what fraction of test points are correctly classified?

Example:

Dataset is 90% positive, 10% negative.

A model that always predicts “positive” gets 90% accuracy!

Is that a good model? *No*. We need better metrics.

Confusion Matrix

		Classifier Prediction	
		Positive	Negative
Actual Value	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

Diagonal = correct, *Off-diagonal* = errors

Key metrics:

- **Recall** = $TP / (TP + FN)$: “Of all actual positives, how many did we catch?”
- **Precision** = $TP / (TP + FP)$: “Of all predicted positives, how many were right?”

Precision vs Recall: A tradeoff

Suppose, you are building a cancer screening test:

	Predicted +	Predicted -
Actual +	80	20
Actual -	10	90

Precision = $80/90 \approx 89\%$. Recall = $80/100 = 80\%$.

Which matters more here: fewer false alarms, or fewer missed cancers?

More metrics

From the confusion matrix:

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

Key rates:

- **True Positive Rate (TPR)** = Recall = Sensitivity

$$TPR = \frac{TP}{TP + FN}$$

- **False Positive Rate (FPR)**

$$FPR = \frac{FP}{FP + TN}$$

- **Specificity** = 1 - FPR

$$Specificity = \frac{TN}{FP + TN}$$

The F1 Score

What if you want a single number that balances precision and recall?

F1 Score: The *harmonic mean* of precision and recall

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Why harmonic mean?

It penalizes extreme imbalance.

Precision	Recall	Arithmetic Mean	F1 Score
0.90	0.90	0.90	0.90
0.99	0.01	0.50	0.02

A model that ignores one metric can't hide behind a high average!

When to use which metric

Scenario	Prioritize	Why?
Cancer Screening	Recall	Don't miss actual cases
Spam Filter	Precision	Don't block legitimate emails
Balanced Importance	F1 Score	Single balanced metric
Understanding errors	Confusion Matrix	See full error breakdown

Your turn!

A model for detecting fraudulent transactions gives these results:

	Predicted: Fraud	Predicted: Legit
Actual: Fraud	45	5
Actual: Legit	30	920

Calculate:

1. Accuracy
2. Precision (for detecting fraud)
3. Recall (for detecting fraud)

Which metric matters most for fraud detection? Why?

Quick review from k-NN

Scenario: You're building a model to predict whether hospital patients will be readmitted within 30 days. You have:

- 10,000 patient records
- 50 features (demographics, diagnoses, medications, lab values)
- 8% of patients are actually readmitted

Discuss with your group:

- How would you split this data for model selection and evaluation?
- Would you use cross-validation? How many folds?
- Which metric(s) should you prioritize? Why?
- Would k-NN be a good choice here? What concerns do you have?

Intro to k-means clustering

From supervised to unsupervised learning

In the last few lectures, we learned supervised learning with k-NN:

Given: data with labels $\{(x_i, y_i)\}$

Goal: predict labels for new data

What if we don't have labels? What can we still learn from data?

Unsupervised learning: discover structure in data without labels.

Today's goal: clustering — grouping similar data points together.

Why Cluster?

When might you want to group data without knowing the groups in advance?

Real-world applications:

Customer segmentation: group customers by purchasing behavior

Document organization: group news articles by topic

Image compression: group similar colors together

Biology: group genes by expression patterns

Anomaly detection: find points that don't fit any group

Similarity and dissimilarity

To cluster, we need to measure how *similar* or *dissimilar* two data points are.

We work with many types of data:

- User records
- Images
- Text documents
- DNA sequences
- Time series
- ...

Key question: How do we define “similar” mathematically?

Dissimilarity as a distance

A *dissimilarity function* $d(x, y)$ returns a large value when x and y are different.

The most common type is a *metric* (also called a *distance function*).

A function $d(x, y)$ is a *metric* if it satisfies:

Property	Meaning
$d(x, y) = 0 \Leftrightarrow x = y$	Zero distance means identical
$d(x, y) \geq 0$	Distances are non-negative
$d(x, y) = d(y, x)$	Symmetric
$d(x, z) \leq d(x, y) + d(y, z)$	Triangle inequality

The triangle inequality

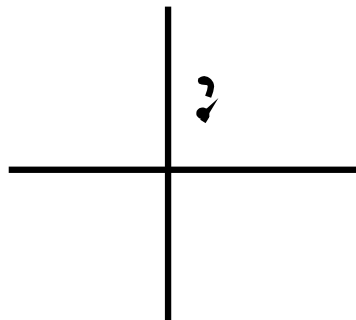
The *triangle inequality* says: the direct path is never longer than going through a third point.



$$d(x, z) \leq d(x, y) + d(y, z)$$


Euclidean distance satisfies all metric properties — it's our go-to dissimilarity measure.

Is "number of different characters" a metric for strings? (e.g., $d(\text{"cat"}, \text{"car"}) = 1$) why/why not?



Cosine similarity

Sometimes we care about *direction*, not magnitude.

Example: Comparing documents by word frequency.

Document 1 (short):

$$\mathbf{x} = \begin{bmatrix} 3 \\ 7 \\ 4 \end{bmatrix} \leftarrow \begin{bmatrix} \text{dog} \\ \text{cat} \\ \text{house} \end{bmatrix}$$

Document 2 (long):

$$\mathbf{y} = \begin{bmatrix} 30 \\ 70 \\ 40 \end{bmatrix}$$

Same topics, just 10x longer!

Euclidean distance says these are very different. But they're about the same topics!

$$\mathbf{x}^T \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos(\theta)$$

Recall: The dot product relates to the angle between vectors:

$$\cos(\theta_{\mathbf{x},\mathbf{y}}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

Rearranging gives us cosine similarity:

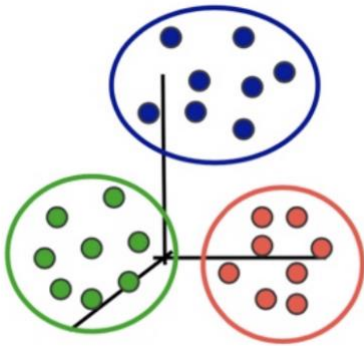
- *cos* = 1: vectors point the same direction (most similar)
- *cos* = 0: vectors are perpendicular (unrelated)
- *cos* = -1: vectors point opposite directions

To convert to *dissimilarity*: use $1 - \cos(\theta)$

What is clustering?

A **clustering** is a grouping of data where:

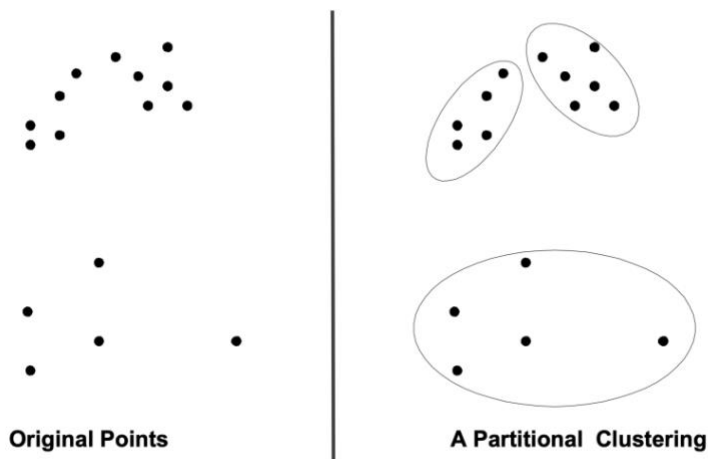
- Points **within** a group are similar
- Points in **different** groups are dissimilar



More formally: minimize intra-cluster distances, maximize inter-cluster distances.

intra = within the cluster
inter = between clusters

We focus on partitional clustering: each point belongs to exactly one cluster, and every point is assigned.



Application: Image compression

Each pixel has a color (R, G, B). With 8 bits per channel, that's $2^{24} \approx 16$ million possible colors.

Idea: Cluster similar colors together, replace each pixel with its cluster's "average" color.

Fewer colors = fewer bits needed = compression!

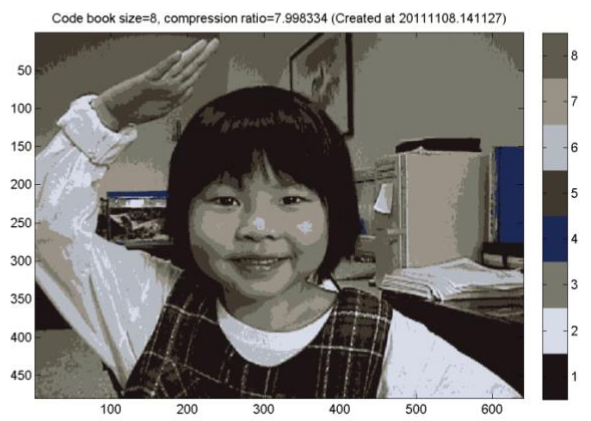


Image compression results

16 colors (4 bits)



8 colors (3 bits)



4 colors (2 bits)



2 colors (1 bit)



The Centroid

To define k -means, we need the *centroid* (also called *center* or *mean*).

Given vectors $\mathbf{x}_1, \dots, \mathbf{x}_n$, the centroid is their *average*:

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$

The centroid is the “center of mass” of the points.

Find the centroid of $\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 6 \\ 2 \end{bmatrix}$

The k – means problem

Given:

- Data points $\underline{x_1, \dots x_n}$ in $\mathbb{R}^n$
- Number of clusters $\underline{k}$

Find:

k centroids $\underline{c_1, \dots c_k}$ that minimize the *cost function*:

$$\text{Cost} = \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{c}_{j(i)}\|^2$$

where $j(i)$ is the index of the centroid closest to $\mathbf{x}_i$.

In other words: *minimize the total squared distance from each point to its nearest center.*

The k – means algorithm

How hard is this?

Some special cases are easy:

- $k = 1$: centroid is just the mean of all points
- $k = n$: each point is its own cluster

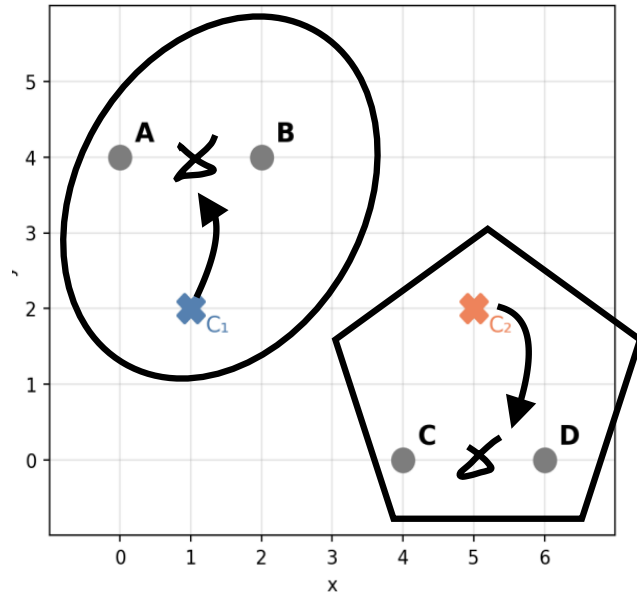
But in general? The problem is *NP-hard*— no efficient exact algorithm exists.

The good news: there's a simple algorithm that works well in practice. It's one of the top 10 algorithms in data mining!

The algorithm

- *Initialize*: Pick k starting centers (randomly or strategically)
- *Assign*: Put each point in the cluster of its nearest center
- *Update*: Recompute each center as the centroid of its cluster
- *Repeat* steps 2-3 until nothing changes

Practice: k – means by hand



Data points:

$A = (0, 4)$, $B = (2, 4)$, $C = (4, 0)$, $D = (6, 0)$

Initial centers:

$C1 = (1, 2)$, $C2 = (5, 2)$

Work through one iteration:

- Assign each point to its nearest center
- Compute new center positions
- Sketch the new centers on the plot

The clustering workflow

A typical clustering project follows these steps:

- ***Prepare data***: clean, normalize, select features
- ***Choose k*** : how many clusters? (more on this next lecture)
- ***Run algorithm***: fit k-means to the data
- ***Visualize***: do the clusters make sense?
- ***Interpret***: what does each cluster represent?
- ***Validate***: are the clusters meaningful?

Our Dataset: 30 dimensions

Visualizing the dataset

Suppose we have a dataset that contains 100 data points, each with 30 features.

```
1 X, y_true = sk_data.make_blobs(n_samples=100, centers=3, n_features=30,  
2                               center_box=(-10.0, 10.0), random_state=0)  
3 print(f"Shape: {X.shape}")  
4 print(f"100 points, each with 30 features")
```

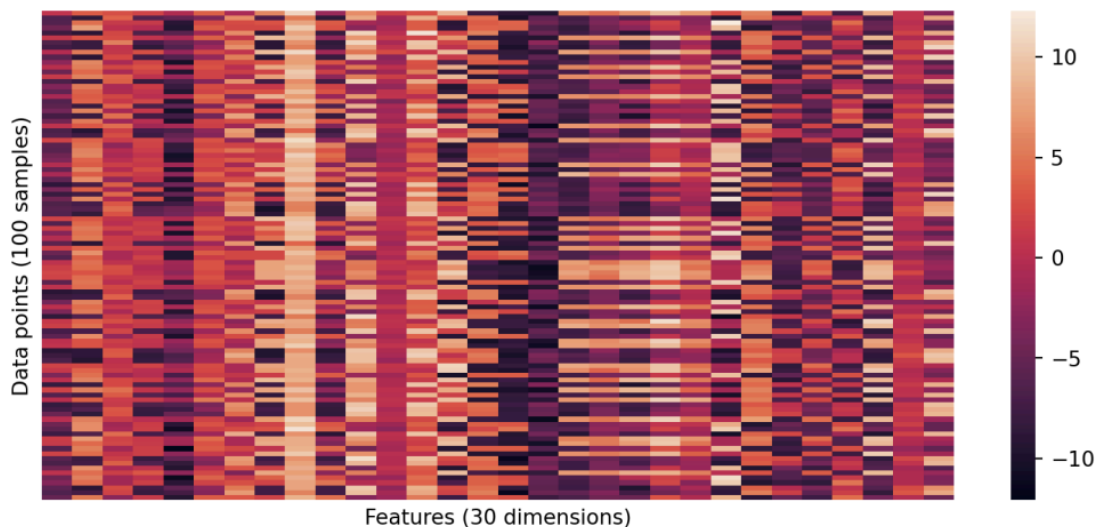
Shape: (100, 30)

100 points, each with 30 features

How do you visualize 30 dimensions? You can't plot it directly!

One option: *Heatmap!*

Each row is a point, each column is a feature.



Do you see any clusters here? What makes it so hard to see?

Our Dataset: 30 dimensions

Visualizing the dataset

Pairwise distances

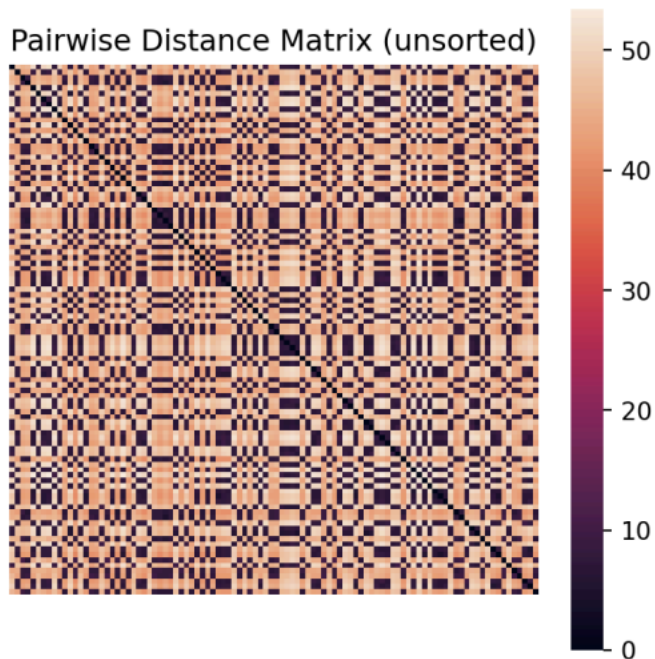
Another view: compute the distance between every pair of points.

```
1 euclidean_dists = metrics.euclidean_distances(X)
2 print(f"Distance matrix shape: {euclidean_dists.shape}")
3 print(f"Distance from point 0 to point 1: {euclidean_dists[0, 1]:.2f}")
```

Distance matrix shape: (100, 100)

Distance from point 0 to point 1: 47.74

This gives us a 100×100 matrix. Entry (i, j) = distance between points i and j .



Our Dataset: 30 dimensions

The visualization problem

Heatmaps show raw values or distances, but hard to interpret.

Can we find 2D coordinates that ***preserve*** the distances between points?

This is exactly what dimensionality-reduction tools like ***Multidimensional Scaling (MDS), t-SNE, and UMAP do!***

Our Dataset: 30 dimensions

Multidimensional Scaling (MDS)

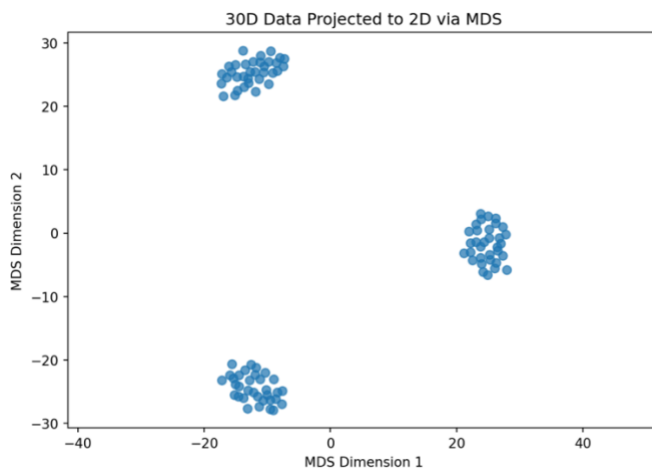
Goal: Find 2D coordinates where distances match the original high dimensional distances.

How it works:

- Start with random 2D positions
- Compare distances in 2D to true distances in 30D
- Adjust positions to reduce the mismatch
- Repeat until distances are as close as possible

In practice, t-SNE and UMAP are more common, but the algorithms are more complex.

```
1 mds = sklearn.manifold.MDS(n_components=2, max_iter=3000, eps=1e-9
2                             random_state=0, dissimilarity="precomp
3 pos = mds.fit_transform(euclidean_dists)
```



Running *k*-means

scikit-learn provides `KMeans` in `sklearn.cluster`:

```
1 kmeans = KMeans(n_clusters=3, init='k-means++', n_init=10, random_state=0)
2 kmeans.fit(X);
```

Key parameters:

- *n_clusters*: the *k* in *k*-means
- *init='k-means++'*: smart initialization (more next lecture)
- *n_init*: run algorithm multiple times, keep best result

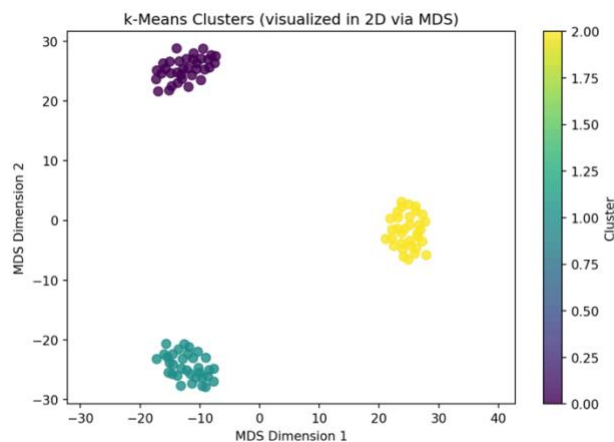
Examining the results

```
1 # Cluster assignments for each point
2 labels = kmeans.labels_
3 print(f"Labels: {labels}")
4
5 # Cluster centers (30-dimensional vectors)
6 print(f"\nCentroid shapes: {kmeans.cluster_centers_.shape}")
7
8 # Total within-cluster sum of squares (the cost function)
9 print(f"\nInertia (cost): {kmeans.inertia_:.1f}")
```

```
Labels: [0 1 2 2 1 0 1 1 0 1 2 0 2 0 1 2 2 0 2 1 2 1 2 0 1 1 0 2 2 2 0 2 0 2 1 2
 1 2 1 1 1 0 2 0 2 0 1 0 2 2 0 0 0 0 2 0 1 1 2 0 2 1 0 0 1 2 0 0 1 1 1 0 0
 2 0 1 0 1 0 1 1 1 0 2 2 1 0 0 2 0 1 1 2 0 2 2 1 1 2]
```

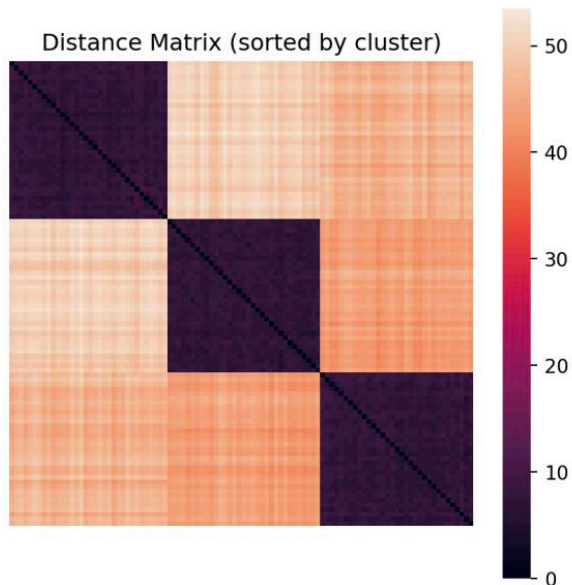
Centroid shapes: (3, 30)

Inertia (cost): 2733.8



Sorted distance matrix

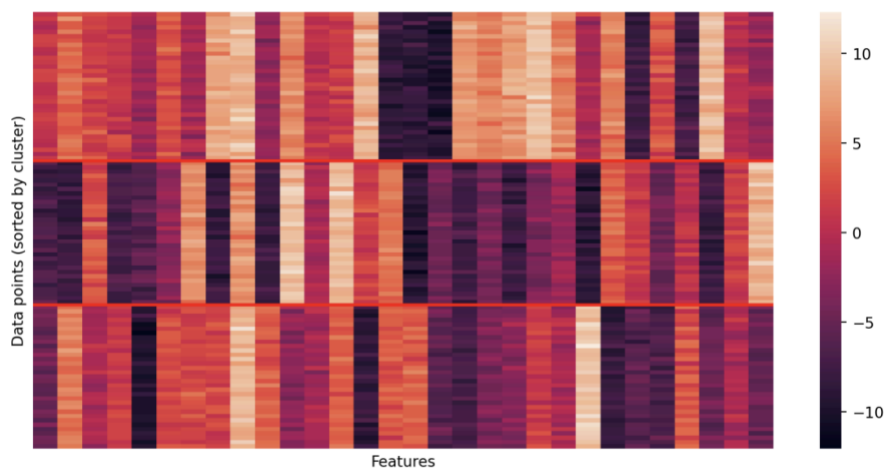
Let's reorder points by cluster and look at the distance matrix again:



Dark blocks on diagonal = points in same cluster are close. Success!

Sorted heatmap

Same idea for the raw data — sort rows by cluster:



Now we can see that clusters have different feature patterns!

Interpreting clusters

We found 3 clusters — but what do they mean?

For *synthetic data*: we know the “true” labels, so we can check.

For *real data*: we need to examine the clusters to understand them:

- Look at cluster centroids (what’s the “average” member like?)
- Look at representative examples from each cluster
- Compare feature distributions across clusters
- Use domain knowledge to assign meaningful labels

Our synthetic data has true labels `y_true`. Let’s compare:

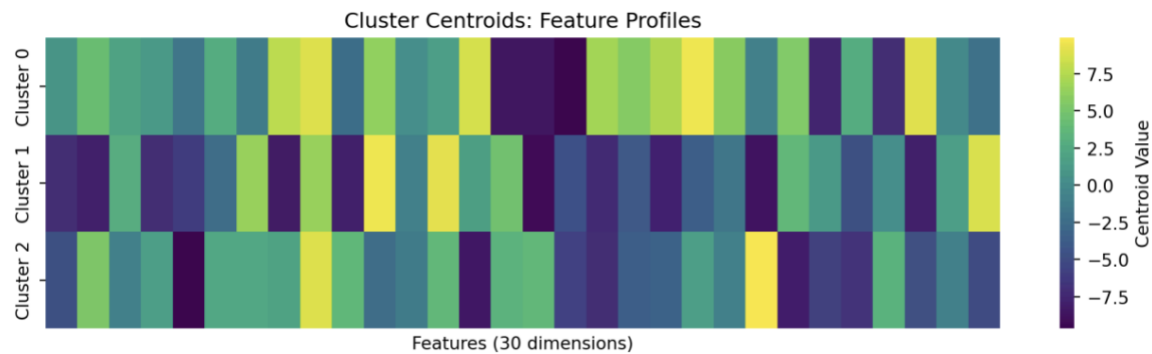
```
1 # Create a cross-tabulation
2 import pandas as pd
3 comparison = pd.crosstab(y_true, labels,
4                           rownames=['True Label'],
5                           colnames=['k-Means Label'])
6 print(comparison)
```

k-Means Label	0	1	2
True Label			
0	34	0	0
1	0	0	33
2	0	33	0

Why are the labels "permuted"? Does k-means cluster 0 have to correspond to true label 0? Why or why not?

Cluster Profiles

What makes each cluster different? Look at the centroids:



Each row shows a cluster's “signature” — distinct patterns across features.

Practice: Interpret these clusters:

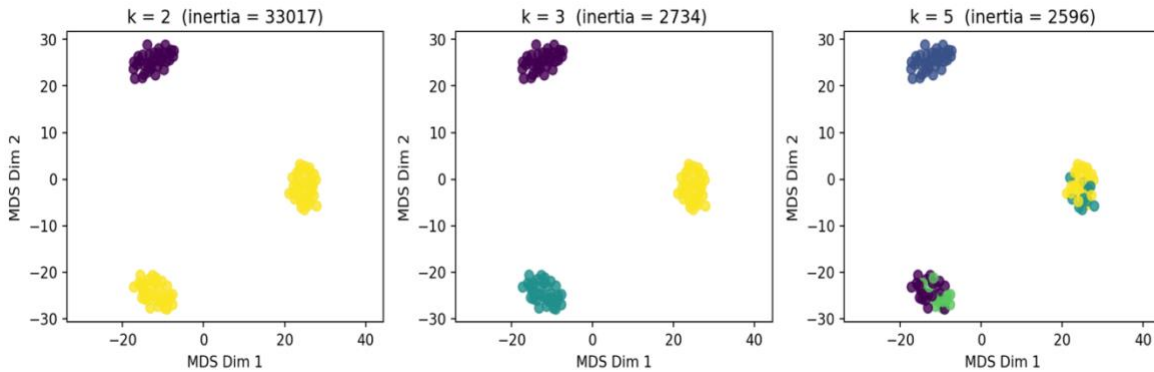
A retailer clustered their customers and got these centroids:

	Age	Income (\$K)	Monthly Spend (\$)	Visits/Month
Cluster 0	42	55	150	8
Cluster 1	28	95	420	3
Cluster 2	58	140	180	2

- Give each cluster a descriptive name
- What marketing strategy might work for each?

What if we pick the wrong k ?

We used $k = 3$ because we generated 3 clusters. But what if we didn't know?



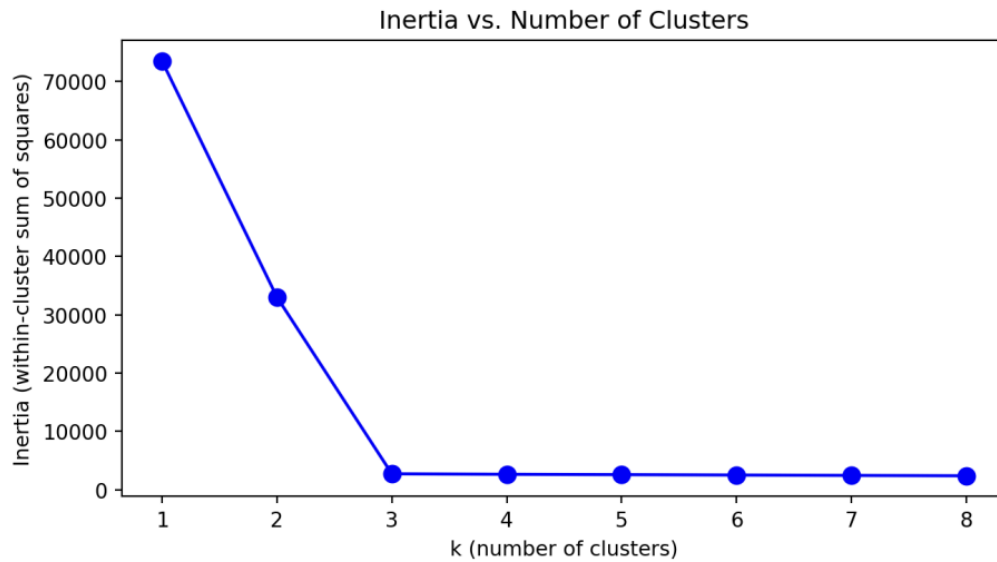
Look at the plots:

- $k = 2$: Two real clusters got merged. Which ones?
- $k = 5$: Some real clusters got split. Does this seem useful?
- The inertia (cost) decreases as k goes up. Why?

Lower inertia doesn't always mean a better model — we need smarter ways to choose k !

Preview: How inertia changes with k

Predict: Where is the "best" k on this plot? How would you decide?



Workflow Summary

Clustering Workflow:

- *Visualize:* Use MDS or similar to see if clusters exist
- *Cluster:* Run k-means with chosen k
- *Validate:* Check distance matrix, compare to ground truth if available
- *Interpret:* Examine centroids and cluster members
- *Iterate:* Try different k values, compare results

